LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA
MODUL 5 ANALISIS KOMPLEKSITAS ALGORITMA

Disusun oleh :

Nama : Khaira ulfia
Nim : 2025573010112
Kelas : TI 1C

Dosen pengampu : Muhammad Reza Zulman, S.ST., M.Sc

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE

2026/202

KATA PENGANTAR

Puji syukur kehadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum dengan judul “Analisis Kompleksitas Algoritma.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas pada mata kuliah Praktikum Algoritma dan Struktur Data. Dalam laporan ini dibahas mengenai konsep dasar kompleksitas algoritma, implementasi kode, serta analisis performa dari beberapa algoritma yang telah diuji.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, oleh karena itu kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa yang akan datang. Semoga laporan ini dapat memberikan manfaat bagi pembaca.

Lhokseumawe, 24 april 2026

Khaira Ulfia

penulis

DAFTAR ISI

KATA PENGANTAR 2
DAFTAR ISI 3
BAB I i
PENDAHULUAN i
1.1Latar belakang i
1.2Rumusan masalah i
1.3Tujuan Praktikum i
BAB II 2
LANDASAN TEORI 2
2.1 Pengertian Big O Notation 2
2.2 Time Complexity 2
2.3 Space Complexity 2
2.4 Jenis-Jenis Kompleksitas 2
2.5 Konsep Efisiensi Algoritma 2
BAB III 3
METODOLOGI PRAKTIKUM 3
3.1Alat dan Bahan 3
3.2Langkah – langkah 3
BAB IV 10
KESIMPULAN 10
4.1Hasil Praktikum 10
4.2Pembahasan 10
BAB V i
PENUTUP i
5.1Kesimpulan i
DAFTAR PUSTAKA i

BAB I

PENDAHULUAN

1.1Latar belakang

Dalam dunia pemrograman, efisiensi algoritma menjadi aspek yang sangat penting dalam menentukan performa suatu program. Algoritma yang baik tidak hanya menghasilkan output yang benar, tetapi juga mampu menyelesaikan masalah dengan waktu dan penggunaan memori yang optimal. Seiring dengan meningkatnya jumlah data yang diproses, kebutuhan akan algoritma yang efisien menjadi semakin krusial.

Big O Notation digunakan sebagai alat untuk mengukur kompleksitas suatu algoritma, baik dari segi waktu (time complexity) maupun ruang (space complexity). Dengan memahami konsep ini, seorang programmer dapat membandingkan beberapa solusi dan memilih algoritma yang paling efisien. Oleh karena itu, praktikum ini dilakukan untuk memahami dan menganalisis kompleksitas algoritma melalui implementasi kode secara langsung.

1.2Rumusan masalah

Apa yang dimaksud dengan Big O Notation?
Bagaimana cara menentukan time complexity dan space complexity?
Bagaimana perbedaan performa antara algoritma yang berbeda?
Algoritma mana yang lebih efisien berdasarkan hasil praktikum?

1.3Tujuan Praktikum

Memahami mengapa analisis efisiensi algoritma itu penting.
Menjelaskan konsep Time Complexity dan Space Complexity.
Membaca dan menulis notasi Big O: O(1), O(log n), O(n), O(n log n), O(n^2).
Menganalisis kompleksitas loop tunggal dan nested loop .
Membandingkan efisiensi dua algoritma secara empiris menggunakan Date.now().
Mengenali pola kode yang memiliki kompleksitas buruk dan cara memperbaikinya.

BAB II

LANDASAN TEORI

2.1 Pengertian Big O Notation

Big O Notation adalah notasi matematis yang digunakan untuk menggambarkan tingkat pertumbuhan suatu algoritma terhadap ukuran input. Notasi ini membantu dalam memahami seberapa cepat waktu eksekusi atau penggunaan memori meningkat ketika input bertambah.

2.2 Time Complexity

Time complexity adalah ukuran yang menunjukkan berapa lama waktu yang dibutuhkan oleh suatu algoritma untuk menyelesaikan suatu masalah berdasarkan ukuran input. Semakin kecil kompleksitasnya, maka semakin cepat algoritma tersebut.

Contoh:

$O(1) \rightarrow$ konstan
$O(n) \rightarrow$ linear
$O(n^2) \rightarrow$ kuadratik

2.3 Space Complexity

Space complexity adalah ukuran penggunaan memori oleh suatu algoritma selama proses eksekusi. Algoritma yang efisien tidak hanya cepat tetapi juga hemat memori.

2.4 Jenis-Jenis Kompleksitas

1. $O(1)$ waktu tetap
2. $O(\log n)$ logaritmik
3. $O(n)$ linear
4. $O(n \log n)$ linearithmic
5. $O(n^2)$ kuadratik
6. $O(n^3)$ kubik

2.5 Konsep Efisiensi Algoritma

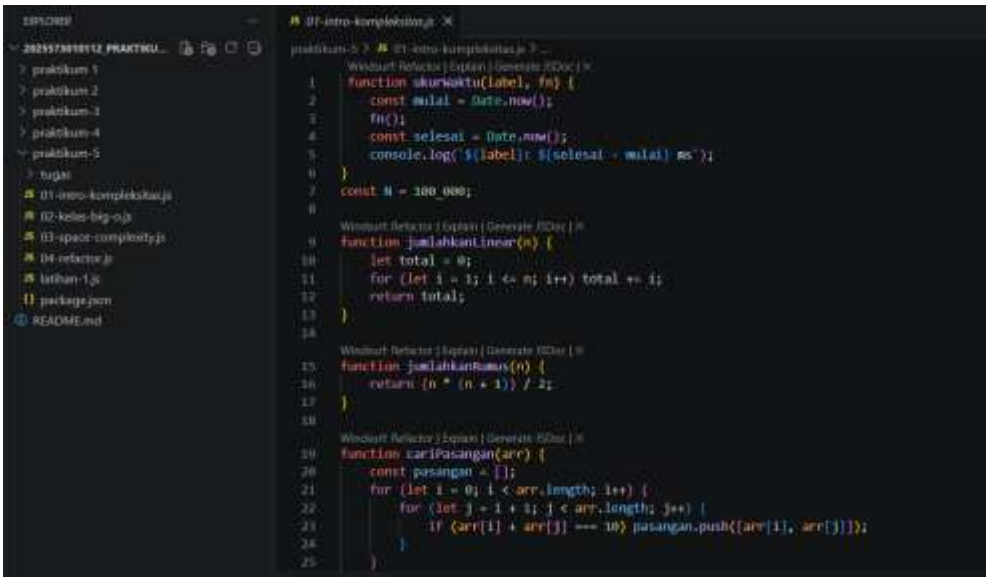
Efisiensi algoritma berkaitan dengan bagaimana suatu algoritma menggunakan sumber daya seperti waktu dan memori. Dalam praktiknya, sering terjadi trade-off antara kecepatan dan penggunaan memori.

BAB III

METODOLOGI PRAKTIKUM

3.1Alat dan Bahan
Laptop/PC
Node.js
Text Editor (VS Code)

3.2Langkah – langkah
1. Buka terminal, buat folder praktikum-5 masuk ke dalam folder itu kemudian inisialisasi npm project pake npm init –y.
Dalam folder praktikum-5 buat file 01-intro-kompleksitas.js, lalu ketik kode yang ada di modul.



Simpan file, outputnya akan seperti ini.

2. buat file 02-kelas-big-o.js, ketik kode yang di modul.

```
EXPLORER
2025573010112_PRAKTIKU...
> praktikum-1
> praktikum-2
> praktikum-3
> praktikum-4
v praktikum-5
  > tugas
  01-intro-kompleksitas.js
  02-kelas-big-o.js
  03-space-complexity.js
  04-refactor.js
  latihan-1.js
  package.json
  README.md

02-kelas-big-o.js
praktikum-5 > 02-kelas-big-o.js > ...
function ambilPertama(arr)  { return arr[0]; }
function tambahkanItem(arr, item) { arr.push(item); }
function isGenap(n) { return n % 2 === 0; }

Windsurf: Refactor | Explain | Generate JSDoc | X
function binarySearch(arr, target) {
    let kiri = 0, kanan = arr.length - 1;
    let langkah = 0;
    while (kiri <= kanan) {
        langkah++;
        const tengah = Math.floor((kiri + kanan) / 2);
        if (arr[tengah] === target) {
            console.log(`  Ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
            return tengah;
        }
        if (arr[tengah] < target) kiri = tengah + 1;
        else kanan = tengah - 1;
    }
    return -1;
}

Windsurf: Refactor | Explain | Generate JSDoc | X
function cariMax(arr) {
    let maks = arr[0];
    for (const x of arr) if (x > maks) maks = x;
    return maks;
}

    const a = [...arr];
    for (let i = 0; i < a.length; i++) {
        for (let j = 0; j < a.length - i - 1; j++)
            if (a[j] > a[j+1]) [a[j], a[j+1]] = [a[j+1], a[j]];
    return a;
}
```

Simpan file, outputnya akan seperti ini.

```
acer@DESKTOP-OK72M54 MINGW64 /d/2025573010112_praktikum_algoritma_dan_struktur_data/praktikum-5 (main)
$ node 02-kelas-big-o.js
=== O(1) — selalu cepat ===
10
true

=== O(log n) — Binary Search ===
  Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) — Linear Search ===
Max dari 1000 elemen: 999

=== O(n²) — Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

=== O(2ⁿ) — Fibonacci Rekursif (n kecil!) ===
0 1 1 2 3 5 8 13 21 34 55

Waktu fib(35) O(2ⁿ):
154 ms
Waktu fib(35) O(n) memoization:
1 ms
```

3. buat file 03-space-complexity, ketik kode yang di modul.

```
EXPLORER
2025573010112_PRAKTIKU...
  praktikum 1
  praktikum 2
  praktikum-3
  praktikum-4
  praktikum-5
    bugus
    01-intro-kompleksitas.js
    02-kelas-big-o.js
    03-space-complexity.js
    04-refactor.js
    latihan-1.js
    package.json
  README.md

02-kelas-big-o.js   03-space-complexity.js
praktikum-5 > 03-space-complexity.js > ...
Windsurf: Refactor | Explain | Generate JSDoc | X
function jumlahArray(arr) {
    let total = 0;
    for (const x of arr) total += x;
    return total;
}

Windsurf: Refactor | Explain | Generate JSDoc | X
function duplikasiArray(arr) {
    const baru = [];
    for (const x of arr) baru.push(x * 2);
    return baru;
}

Windsurf: Refactor | Explain | Generate JSDoc | X
function faktorialRekursif(n) {
    if (n <= 1) return 1;
    return n * faktorialRekursif(n - 1);
}

Windsurf: Refactor | Explain | Generate JSDoc | X
function faktorialIteratif(n) {
    let hasil = 1;
    for (let i = 2; i <= n; i++) hasil *= i;
    return hasil;
}

const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
```

Simpan file, outputnya akan seperti ini.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS

acer@DESKTOP-OK72M54 MINGW64 /d/2025573010112_praktikum_algoritma_dan_struktur_data/praktikum-5 (main)
$ node 03-space-complexity.js
Jumlah array   : 55
Duplikasi array: [
   2,  4,  6,  8, 10,
  12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7
Lambat: [ 2, 7 ]
Cepat: [ 2, 7 ]
Waktu Lambat: 1044 ms
Waktu Cepat: 6 ms
```

4. buat file 04-refactor.js, lalu ketik kode yang di modul.

```
EXPLORER
2025573010112_PRAKTIKUM_ALGORITMA_DAN_...
> praktikum 1
> praktikum 2
> praktikum-3
> praktikum-4
v praktikum-5
  > tugas
  JS 01-intro-kompleksitas.js
  JS 02-kelas-big-o.js
  JS 03-space-complexity.js
  JS 04-refactor.js
  JS latihan-1.js
  {} package.json
  (i) README.md

JS 02-kelas-big-o.js   JS 03-space-complexity.js   JS 04-refactor.js ×
praktikum-5 > JS 04-refactor.js > ...
Windsurf: Refactor | Explain | Generate JSDoc | ×
function adaDuplikatLambat(arr) {
    for (let i = 0; i < arr.length; i++)
        for (let j = i+1; j < arr.length; j++)
            if (arr[i] === arr[j]) return true;
    return false;
}

Windsurf: Refactor | Explain | Generate JSDoc | ×
function adaDuplikatCepat(arr) {
    const seen = new Set();
    for (const x of arr) {
        if (seen.has(x)) return true;
        seen.add(x);
    }
    return false;
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function frekuensiLambat(arr) {
    const unik = [];
    const hitung = [];
    for (const x of arr) {
        const idx = unik.indexOf(x);
        if (idx === -1) { unik.push(x); hitung.push(1); }
        else hitung[idx]++;
    }
    return Object.fromEntries(unik.map((u,i) => [u, hitung[i]]));
}
Windsurf: Refactor | Explain | Generate JSDoc | ×
function frekuensiCepat(arr) {
```

Simpan file, outputnya akan seperti ini.

```
PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS
$ node 03-space-complexity.js

acer@DESKTOP-OK72M54 MINGW64 /d/2025573010112_praktikum_algoritma_dan_struktur_data/praktikum-5 (main)
$ node 04-refactor.js
Duplikat LAMBAT O(n²): 0 ms
Duplikat CEPAT  O(n) : 0 ms

Frekuensi: { a: 3, b: 2, c: 1, d: 1 }

acer@DESKTOP-OK72M54 MINGW64 /d/2025573010112_praktikum_algoritma_dan_struktur_data/praktikum-5 (main)
$
```

Latihan :

Tambahin kode di bawah file 04-space-complexity.js

```
function fnA(n) {
    return n * 2;
}

function fnB(n) {
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
        }
    }
}

function fnC(n) {
    for (let i = 1; i < n; i *= 2) {
    }
}

function fnD(arr) {
    arr.forEach(x => {
        arr.forEach(y => {
            arr.forEach(z => {
            });
```

Simpan, jalankan dari terminal outputnya akan seperti ini.

```
acer@DESKTOP-OK72M54 MINGW64 /d/2025573010112_praktikum_algoritma_dan_struktur_data/praktikum-5 (main)
$ node latihan-1.js
fnA → waktu: 0 ms
fnB → waktu: 4 ms
fnC → waktu: 0 ms
fnD → waktu: 3 ms

acer@DESKTOP-OK72M54 MINGW64 /d/2025573010112_praktikum_algoritma_dan_struktur_data/praktikum-5 (main)
$
```

Tugas 1

```
// Fungsi A: Intersection dari dua array
// Versi O(n²) menggunakan nested loop
function intersectionO_n2(arr1, arr2) {
  let result = [];
  for (let i = 0; i < arr1.length; i++) {
    for (let j = 0; j < arr2.length; j++) {
      if (arr1[i] === arr2[j] && !result.includes(arr1[i])) {
        result.push(arr1[i]);
      }
    }
  }
  return result;
}

// Versi O(n) menggunakan Set
function intersectionO_n(arr1, arr2) {
  let set2 = new Set(arr2);
  let result = new Set();

  for (let i = 0; i < arr1.length; i++) {
    if (set2.has(arr1[i])) {
      result.add(arr1[i]);
    }
  }

  return Array.from(result);
}
```

Outputnya seperti ini

```
acer@DESKTOP-OK72M54 MINGW64 /d/2025573010112_praktikum_algoritma_dan_struktur_data/praktikum-5 (main)
$ cd tugas

acer@DESKTOP-OK72M54 MINGW64 /d/2025573010112_praktikum_algoritma_dan_struktur_data/praktikum-5/tugas (main)
$ node tugas-1.js
=== Fungsi A: Intersection ===
O(n²): [ 3, 4 ]
O(n): [ 3, 4 ]

=== Fungsi B: Group Anagrams ===
[ [ 'eat', 'tea', 'ate' ], [ 'tan', 'nat' ], [ 'bat' ] ]

=== Fungsi C: Perfect Square Sum ===
O(n³): true
O(n log n): true
```

Tugas 2

```javascript
// Fungsi untuk mengukur waktu eksekusi
function benchmarkSemua(ukuranData) {
  console.log(`\n=== Benchmark dengan ukuran data: ${ukuranData} ===\n`);

  // Jalankan setiap fungsi dan catat waktu eksekusinya
  for (let ukuran of ukuranData) {
    console.log(`--- Ukuran: ${ukuran} ---`);

    // Test fn_O1
    let start = performance.now();
    fn_O1(ukuran);
    let end = performance.now();
    console.log(`fn_O1(n): ${(end - start).toFixed(4)} ms`);

    // Test fn_On
    start = performance.now();
    fn_On(ukuran);
    end = performance.now();
    console.log(`fn_On(n): ${(end - start).toFixed(4)} ms`);

    // Test fn_OnLogn
    start = performance.now();
    fn_OnLogn(ukuran);
    end = performance.now();
    console.log(`fn_OnLogn(n): ${(end - start).toFixed(4)} ms`);

    // Test fn_On2
    start = performance.now();
    fn_On2(ukuran);
    end = performance.now();
    console.log(`fn_On2(n): ${(end - start).toFixed(4)} ms\n`);
  }
```

Outputnya seperti ini.

```
acer@DESKTOP-OK72M54 MINGW64 /d/2025573010112_praktikum_algoritma_dan_struktur_data/praktikum-5/tugas (main)
$ node tugas-2.js

=== Benchmark dengan ukuran data: 100,500,1000,5000,10000 ===

--- Ukuran: 100 ---
fn_O1(n): 0.0109 ms
fn_On(n): 0.0887 ms
fn_OnLogn(n): 0.0682 ms
fn_On2(n): 0.4965 ms

--- Ukuran: 500 ---
fn_O1(n): 0.0028 ms
fn_On(n): 0.0096 ms
fn_OnLogn(n): 0.2779 ms
fn_On2(n): 1.8701 ms

--- Ukuran: 1000 ---
fn_O1(n): 0.0021 ms
fn_On(n): 0.0385 ms
fn_OnLogn(n): 0.1672 ms
fn_On2(n): 1.9850 ms

--- Ukuran: 5000 ---
fn_O1(n): 0.0008 ms
fn_On(n): 0.0134 ms
fn_OnLogn(n): 0.9311 ms
fn_On2(n): 80.4193 ms

--- Ukuran: 10000 ---
fn_O1(n): 0.0032 ms
fn_On(n): 0.0403 ms
fn_OnLogn(n): 3.0528 ms
fn_On2(n): 263.1246 ms
```

BAB IV

KESIMPULAN

4.1Hasil Praktikum

Hasil praktikum menunjukkan bahwa setiap algoritma memiliki waktu eksekusi yang berbeda tergantung kompleksitasnya. Algoritma dengan kompleksitas lebih rendah cenderung lebih cepat dibandingkan algoritma dengan kompleksitas tinggi, terutama saat ukuran data besar.

4.2Pembahasan

Berdasarkan hasil pengujian, algoritma dengan kompleksitas waktu O(1) dan O(n) menunjukkan performa yang sangat cepat dan stabil. Hal ini karena jumlah operasi yang dilakukan tidak meningkat secara signifikan terhadap ukuran input. Sebaliknya, algoritma dengan kompleksitas $O(n^2)$ dan $O(n^3)$ mengalami peningkatan waktu eksekusi yang drastis ketika ukuran input diperbesar.

Pada percobaan pencarian pasangan angka, fungsi dengan metode nested loop ($O(n^2)$) terbukti lebih lambat dibandingkan metode menggunakan Set (O(n)). Hal ini menunjukkan bahwa penggunaan struktur data tambahan dapat meningkatkan efisiensi waktu, meskipun membutuhkan ruang memori lebih besar.

Dengan demikian, pemilihan algoritma sangat bergantung pada kebutuhan sistem. Jika prioritas adalah kecepatan, maka algoritma dengan kompleksitas lebih rendah harus dipilih. Namun, jika memori terbatas, maka perlu mempertimbangkan trade-off antara time dan space complexity.

BAB V

PENUTUP

5.1Kesimpulan

Berdasarkan praktikum yang telah dilakukan mengenai analisis kompleksitas algoritma, dapat disimpulkan bahwa Big O Notation merupakan metode yang digunakan untuk mengukur efisiensi suatu algoritma dalam hal waktu dan ruang. Setiap algoritma memiliki tingkat kompleksitas yang berbeda-beda tergantung pada jumlah operasi yang dilakukan terhadap ukuran input.

Selain itu, algoritma dengan kompleksitas rendah seperti O(1) dan O(n) terbukti lebih efisien dibandingkan algoritma dengan kompleksitas tinggi seperti $O(n^2)$ atau $O(n^3)$, terutama ketika digunakan pada data berukuran besar. Hasil pengujian juga menunjukkan bahwa penggunaan struktur data tambahan seperti Set dapat mempercepat proses, meskipun membutuhkan memori lebih besar.

Dengan demikian, pemilihan algoritma yang tepat sangat penting untuk meningkatkan performa program. Pemahaman terhadap time complexity dan space complexity membantu dalam menentukan solusi yang optimal sesuai dengan kebutuhan sistem.

DAFTAR PUSTAKA

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data Structures and Algorithms in Java (6th ed.). Wiley.

Knuth, D. E. (1997). The Art of Computer Programming. Addison-Wesley.

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

Weiss, M. A. (2013). Data Structures and Algorithm Analysis in Java (3rd ed.). Pearson.